

PROMT PROPORCIONADO:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python. Contexto: Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. El proyecto ya permite convertir una expresión infija a posfija utilizando una clase Stack creada en clase. Objetivo: Necesito complementar el proyecto para evaluar una expresión posfija utilizando la misma clase Stack. En esta práctica se usarán expresiones numéricas con operandos de un solo dígito, por ejemplo $2 + 3 * 4$, cuya expresión posfija sería $234*+$. Instrucciones importantes: 1. No generes un proyecto nuevo desde cero. 2. No reemplaces la clase Stack existente. 3. No uses eval() de Python. 4. Respeta la organización actual del proyecto. 5. La solución debe estar hecha con programación orientada a objetos. 6. La interfaz gráfica debe seguir usando el diseño creado con Qt Designer. 7. Propón cambios mínimos para agregar la evaluación posfija. 8. Explica en qué archivo y en qué parte debo integrar el código. 9. Explica cómo se usa la pila durante la evaluación. 10. La evaluación debe recorrer la expresión posfija, apilar los dígitos como números y aplicar las operaciones cuando encuentre operadores. 11. Proporciona pruebas para validar el funcionamiento. Código existente:

```
from estructuras.lineales.lista_enlazada_simple import LinkedList class OperacionesPila: def __init__(self, lista: LinkedList): self.lista = lista self.resultado = "" def pila_agregar(self, data): self.lista.insert_at_beginning(data) self.resultado = f"Elemento {data} agregado a la pila." return self.resultado def pila_eliminar(self): eliminado = self.lista.delete_first() if eliminado is None: self.resultado = "La pila está vacía, no se puede eliminar." else: self.resultado = f"Elemento {eliminado} eliminado de la pila." return eliminado def pila_tope(self): if self.lista.head is None: self.resultado = "La pila está vacía." return None self.resultado = f"El tope de la pila es: {self.lista.head.data}" return self.lista.head.data def pila_imprimir(self): if self.lista.head is None: self.resultado = "La pila está vacía." return self.resultado elementos = [] temp = self.lista.head while temp is not None: elementos.append(str(temp.data)) temp = temp.next self.resultado = " -> ".join(elementos) return self.resultado def is_empty(self): return self.lista.head is None from estructuras.lineales.stack import OperacionesPila from estructuras.lineales.lista_enlazada_simple import LinkedList class ConvertidorExpresiones: def __init__(self): self.pila = OperacionesPila(LinkedList()) def prioridad(self, operador): if operador == '$': # Potencia return 3 if operador in ('*', '/'): return 2 if operador in ('+', '-'): return 1 return 0 def infija_a_posfija(self, expresion): salida = [] for token in expresion: if token.isalnum(): salida.append(token) elif token == '(': self.pila.pila_agregar(token) elif token == ')': while not self.pila.is_empty() and self.pila.pila_tope() != '(': salida.append(self.pila.pila_eliminar()) self.pila.pila_eliminar() else: while (not self.pila.is_empty() and self.prioridad(self.pila.pila_tope()) >= self.prioridad(token)): salida.append(self.pila.pila_eliminar()) self.pila.pila_agregar(token) while not self.pila.is_empty(): salida.append(self.pila.pila_eliminar()) return " ".join(salida) #PDF, EXPLICACION, PROMTS, EVIDENCIA import sys from PyQt5.QtWidgets import QApplication from load.load_MenuPrincipal import MenuPrincipal def main(): app = QApplication(sys.argv) ventana = MenuPrincipal() ventana.show() sys.exit(app.exec_()) if __name__ == "__main__": main() from PyQt5.QtWidgets import QDialog from PyQt5 import uic from menus.menu_pila import OperacionesPila from estructuras.lineales.conversion_expresiones import ConvertidorExpresiones class LoadInterfazPila(QDialog): def __init__(self): super().__init__() self.ui = uic.loadUi('ui/conversion_infija_posfija.ui', self) # Ya no usamos LinkedList, solo la pila #self.operaciones = OperacionesPila([]) self.convertidor = ConvertidorExpresiones() # Conectar botón de conversión self.btn_convertir.clicked.connect(self.convertir_infija_a_posfija) def convertir_infija_a_posfija(self): expresion_infija = self.txt_expresion.text() if not expresion_infija.strip():
```

```
self.lbl_resultado.setText("Ingrese una expresión válida.") return try: resultado_posfija =
self.convertidor.infija_a_posfija(expresion_infija) self.lbl_resultado.setText(resultado_posfija) except
Exception as e: self.lbl_resultado.setText(f"Error: {str(e)}")
```

Resultado esperado: - Primero analiza la estructura del código existente. - Después propon la clase o método para evaluar la expresión posfija numérica. - Luego indica cómo conectar ese método con el botón de la interfaz gráfica. - Finalmente proporciona pruebas para comprobar que funciona. - No omitas la explicación, porque debo entender el código para defenderlo en clase.

RESPUESTA POR PARTE DE LA IA:

Perfecto, Alejandro. Vamos a complementar tu proyecto con la **evaluación de expresiones posfijas** usando la misma pila que ya tienes.

Análisis del código existente

- Ya cuentas con:
 - Una clase `OperacionesPila` que encapsula operaciones sobre una lista enlazada usada como pila.
 - Una clase `ConvertidorExpresiones` que convierte de infija a posfija.
 - Una interfaz (`LoadInterfazPila`) que captura la expresión infija y muestra la posfija.

Lo que falta es **evaluar la expresión posfija** y mostrar el resultado en la interfaz.

Clase para evaluar posfija

Agrega un nuevo método dentro de `ConvertidorExpresiones`:

python

```
class ConvertidorExpresiones:
    def __init__(self):
        self.pila = OperacionesPila(LinkedList())

    # ... ya tienes prioridad() e infija_a_posfija()

    def evaluar_posfija(self, expresion_posfija: str):
        # Reiniciar pila
        self.pila = OperacionesPila(LinkedList())

        for token in expresion_posfija.split():
            if token.isdigit(): # Operando
                self.pila.pila_agregar(int(token))
            else: # Operador
                op2 = self.pila.pila_eliminar()
                op1 = self.pila.pila_eliminar()
```

```

        if token == '+':
            self.pila.pila_agregar(op1 + op2)
        elif token == '-':
            self.pila.pila_agregar(op1 - op2)
        elif token == '*':
            self.pila.pila_agregar(op1 * op2)
        elif token == '/':
            self.pila.pila_agregar(op1 // op2)    # División entera
        elif token == '$':
            self.pila.pila_agregar(op1 ** op2)    # Potencia

    return self.pila.pila_eliminar()

```

Dónde integrar el código

- Archivo: estructuras/lineales/conversion_expresiones.py
- Inserta el método `evaluar_posfija` dentro de la clase `ConvertidorExpresiones`.
- No necesitas modificar `OperacionesPila`, solo reutilizarla.

Conexión con la interfaz

En tu clase `LoadInterfazPila`, agrega un botón en el `.ui` (por ejemplo `btn_evaluar`) y conéctalo:

python

```

class LoadInterfazPila(QDialog):
    def __init__(self):
        super().__init__()
        self.ui = uic.loadUi('ui/conversion_infija_posfija.ui', self)
        self.convertidor = ConvertidorExpresiones()

        self.btn_convertir.clicked.connect(self.convertir_infija_a_posfija)
        self.btn_evaluar.clicked.connect(self.evaluar_posfija)

    def convertir_infija_a_posfija(self):
        expresion_infija = self.txt_expresion.text()
        if not expresion_infija.strip():
            self.lbl_resultado.setText("Ingrese una expresión válida.")
            return
        try:
            resultado_posfija =
self.convertidor.infija_a_posfija(expresion_infija)
            self.lbl_resultado.setText(resultado_posfija)
        except Exception as e:
            self.lbl_resultado.setText(f"Error: {str(e)}")

    def evaluar_posfija(self):
        expresion_posfija = self.lbl_resultado.text()
        if not expresion_posfija.strip():
            self.lbl_resultado.setText("Primero convierta la expresión.")
            return
        try:

```

```

        resultado = self.convertidor.evaluar_posfija(expresion_posfija)
        self.lbl_resultado.setText(f"Resultado: {resultado}")
    except Exception as e:
        self.lbl_resultado.setText(f"Error: {str(e)}")

```

⚙️ □ Cómo funciona la pila en la evaluación

1. Se recorre la expresión posfija token por token.
2. Si es un número → se **apila**.
3. Si es un operador → se **desapilan dos operandos**, se aplica la operación y se **apila el resultado**.
4. Al final, el único valor en la pila es el resultado de toda la expresión.

□ Pruebas

- Expresión: $2 + 3 * 4$
 - Infija: $2+3*4$
 - Posfija: $2\ 3\ 4\ *\ +$
 - Evaluación: $2 + (3*4) = 14$
- Expresión: $(5-2)^2$
 - Infija: $(5-2)^2$
 - Posfija: $5\ 2\ -\ 2\ \$$
 - Evaluación: $(5-2)^2 = 9$
- Expresión: $((3+5)*(8-2))/4+7-6$
 - Posfija: $3\ 5\ +\ 8\ 2\ -\ *\ 4\ 2\ \$\ /\ 7\ +\ 6\ -$
 - Evaluación: 4 (como ya comprobamos).